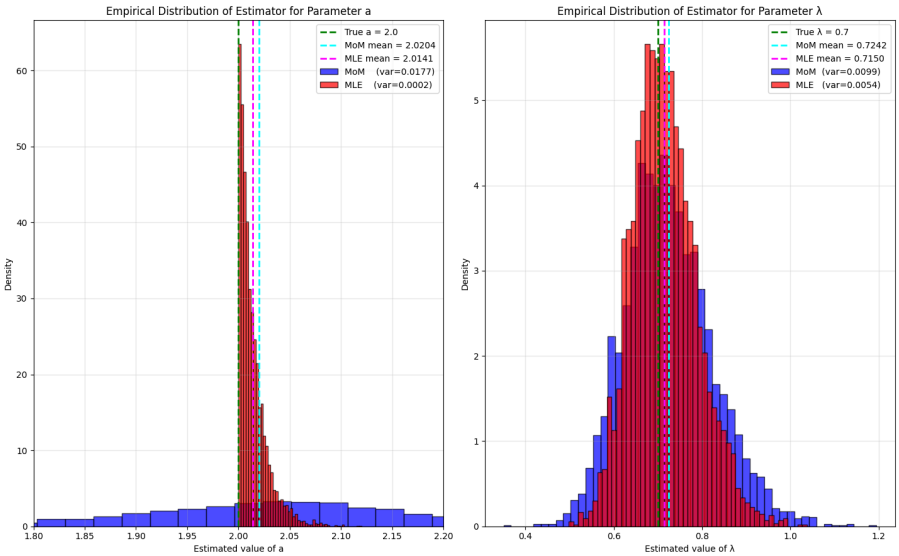
University of Geneva

Faculty of Science

TP D: PCA, k-NN classification

Data Science
14X026



Michel Jean Joseph Donnet

2025-12-08

Contents

Method of Moments, Maximum Likelihood, and Moment Generating Functions

In this assignment we explore three classical estimation techniques on a simple parametric model, and then investigate why higher-order moment matching may fail in practice. The exercises combine theoretical derivations and empirical analysis.

The shifted exponential distribution

Consider the shifted exponential distribution with parameters $a \in \mathbb{R}$ and $\lambda > 0$, with density

$$f_{a,\lambda}(x) = \lambda e^{-\lambda(x-a)} \mathbf{1}_{\{x \geq a\}}.$$

Let $X \sim f_{a,\lambda}$. Throughout the section assume we observe an i.i.d. sample $X_1, ..., X_n$ from this distribution.

Method of Moments

1. Derive the first two theoretical moments of X, i.e. $\mathbb{E}[X]$ and $\mathbb{E}[(X - \mathbb{E}[X])^2]$, and express them as functions of (a, λ).

$$\begin{aligned}
\mathbb{E}[X] &= \int_{-\infty}^{+\infty} x f_{a,\lambda}(x) dx \\
&= \int_{-\infty}^{+\infty} x\lambda e^{-\lambda(x-a)} \mathbf{1}_{\{x \geq a\}} dx \\
&= \int_{a}^{\infty} x\lambda e^{-\lambda(x-a)} dx \\
&= \int_{0}^{\infty} (y+a)\lambda e^{-\lambda y} dy \\
&= \int_{0}^{\infty} y\lambda e^{-\lambda y} dy^{1} + a\int_{0}^{\infty} \lambda e^{-\lambda y} dy \\
&= \left[-ye^{-\lambda y}\right]_0^{\infty} + \int_{0}^{\infty} e^{-\lambda y} dy + a\left[-e^{-\lambda y}\right]_0^{\infty} \\
&= \left(\lim_{y\to\infty} -ye^{-\lambda y} + 0\right) - \left[\frac{1}{\lambda}e^{-\lambda y}\right]_0^{\infty} + a\left(\lim_{y\to\infty} -e^{-\lambda y} + e^{0}\right) \\
&= \left(\lim_{y\to\infty} -\frac{1}{\lambda}e^{-\lambda y} + \frac{1}{\lambda}e^{0}\right) + a \\
&= \frac{1}{\lambda} + a
\end{aligned}$$

So we can derivate the second theoretical moment of X:

[1] $\int udv = uv - \int duv$

$$
\begin{aligned}
\mathbb{E}[(X-\mathbb{E}[X])^2] &= \mathbb{E}\left[\left(X-\frac{1}{\lambda}-a\right)^2\right] \\
&= \int_{-\infty}^{+\infty}\left(x-\frac{1}{\lambda}-a\right)^2 f_{a,\lambda}(x)dx \\
&= \int_{-\infty}^{+\infty}\left(x-a-\frac{1}{\lambda}\right)^2 \lambda e^{-\lambda(x-a)}\mathbf{1}_{\{x\geq a\}}dx \\
&= \int_{a}^{+\infty}\left(x-a-\frac{1}{\lambda}\right)^2 \lambda e^{-\lambda(x-a)}dx \\
&= \int_{0}^{+\infty}\left(y-\frac{1}{\lambda}\right)^2 \lambda e^{-\lambda y}dy \\
&= \int_{0}^{+\infty} y^2\lambda e^{-\lambda y}dy - 2\int_{0}^{+\infty} ye^{-\lambda y}dy + \frac{1}{\lambda}\int_{0}^{+\infty} e^{-\lambda y}dy^{1} \\
&= \left[-y^2e^{-\lambda y}\right]_0^{\infty} + \int_{0}^{+\infty} 2ye^{-\lambda y}dy - \frac{2}{\lambda}\left(\left[-ye^{-\lambda y}\right]_0^{\infty} + \int_{0}^{+\infty} e^{-\lambda y}dy\right) + \frac{1}{\lambda^2} \\
&= \left[-2y\frac{1}{\lambda}e^{-\lambda y}\right]_0^{\infty} + \int_{0}^{+\infty} \frac{2}{\lambda}e^{-\lambda y}dy - \frac{2}{\lambda^2} + \frac{1}{\lambda^2} \\
&= \frac{2}{\lambda^2} - \frac{2}{\lambda^2} + \frac{1}{\lambda^2} \\
&= \frac{1}{\lambda^2}
\end{aligned}
$$

2. Let

$$
\overline{X}_n = \frac{1}{n}\sum_{i=1}^{n} X_i, \quad S_n^2 = \frac{1}{n}\sum_{i=1}^{n}\left(X_i-\overline{X}_n\right)^2
$$

be the empirical mean and (non-unbiased) variance. Set the theoretical moments equal to the empirical ones and solve this system to obtain the moment estimators $\hat{a}_{\text{MoM}}$ and $\hat{\lambda}_{\text{MoM}}$.

[1] $\int_0^{\infty} e^{-\lambda y}dy = \left[-\frac{1}{\lambda}e^{-\lambda y}\right]_0^{\infty} = \left(\lim_{y\to\infty} -\frac{1}{\lambda}e^{-\lambda y} + \frac{1}{\lambda}e^{0}\right) = \frac{1}{\lambda}$

$$\begin{cases} \mathbb{E}[X] = \overline{X_n} \\ \mathbb{E}[(X - \mathbb{E}[X])^2] = S_n^2 \end{cases} = \begin{cases} \frac{1}{\lambda} + a = \overline{X_n} \\ \frac{1}{\lambda^2} = S_n^2 \end{cases}$$

$$= \begin{cases} a = \overline{X_n} - \frac{1}{\lambda} \\ \frac{1}{\lambda^2} - S_n^2 = 0 \end{cases}$$

$$= \begin{cases} a = \overline{X_n} - \frac{1}{\lambda} \\ \left(\frac{1}{\lambda} - S_n\right)\left(\frac{1}{\lambda} + S_n\right) = 0 \end{cases}$$

$$= \begin{cases} a = \overline{X_n} - \frac{1}{\lambda} \\ \lambda = \frac{1}{S_n} \text{ or } \lambda = -\frac{1}{S_n} \end{cases}$$

$$= \begin{cases} a = \overline{X_n} - S_n \text{ or } a = \overline{X_n} + S_n \\ \lambda = \frac{1}{S_n} \qquad\qquad \text{or } \lambda = -\frac{1}{S_n} \end{cases}$$

So we have $\left(\hat{a}_{\text{MoM}}, \hat{\lambda}_{\text{MoM}}\right) = \left(\overline{X_n} - S_n, \frac{1}{S_n}\right)$ and not the other case because the standard deviation S_n is always positive, so λ cannot be $-\frac{1}{S_n}$.

Maximum Likelihood Estimation

3. Write the log-likelihood $\ell(a, \lambda)$ of the sample.

$$\ell(a, \lambda) = \log\left(\prod_{i=0}^{n} f(X_i; a, \lambda)\right)$$

$$= \sum_{i=0}^{n} \log\left(\lambda e^{-\lambda(X_i - a)} \mathbf{1}_{\{X_i \geq a\}}\right)$$

$$= n\log(\lambda) + \sum_{i=0}^{n} -\lambda(X_i - a) \underbrace{\mathbf{1}_{\{X_i \geq a\}}}_{\text{condition on } X_i}$$

4. Show that the MLE of a is $\hat{a}_{\text{MLE}} = \min_i X_i$.

First of all, the log-likelihood is only defined when $X_i \geq a$.

So a must be less or equal to all X_i.

On top of that, we try to maximise the log-likelihood, and the only place where a appears is inside the term $-\lambda(X_i - a)$ which has a negative sign.

So we must minimize the term $-\lambda(X_i - a)$. To do that, a must be maximum. However, a must be less or equal to all X_i. So if we take $a = \min X_i$, the condition is respected and a is maximum since all the X_i are greater or equal to 0. Effectively, they are all probabilities, so between 0 and 1.

So to maximise the log-likelihood, we must take $\hat{a}_{\text{MLE}} = \min_i X_i$.

5. Derive the corresponding MLE $\hat{\lambda}_{\text{MLE}}$ by maximizing the log-likelihood w.r.t. λ.

We want to find the maximum of the log-likelihood function.

To do that, we will compute the derivate of this function according to λ to find the optimum of the function.

$$\frac{\partial \ell(a,\lambda)}{\partial \lambda} = 0 \iff \frac{\partial}{\partial \lambda}\left(n\log(\lambda) + \sum_{i=0}^{n} -\lambda(X_i - \hat{a}_{\text{MLE}})\right) = 0$$
$$\iff \frac{n}{\lambda} - \sum_{i=0}^{n}(X_i - \hat{a}_{\text{MLE}}) = 0$$
$$\iff n - \lambda\sum_{i=0}^{n}(X_i - \hat{a}_{\text{MLE}}) = 0$$
$$\iff \hat{\lambda}_{\text{MLE}} = \frac{n}{\sum_{i=0}^{n}(X_i - \hat{a}_{\text{MLE}})}$$

Then, to check that this stationary point found is a maximum, we need to look at the second derivate of the log-likelihood according to λ, which give us:

$$\frac{\partial^2 \ell(a,\lambda)}{\partial \lambda^2} = \frac{\partial}{\partial \lambda}\left(\frac{n}{\lambda} - \sum_{i=0}^{n}(X_i - \hat{a}_{\text{MLE}})\right)$$
$$= -\frac{n}{\lambda^2}$$

As the second derivate is always negative since n and λ are by definition positive numbers, this means that the log-likelihood function is concave, so the stationary point found is a maximum.

Simulation study

Let $a^\star = 2$ and $\lambda^\star = 0.7$.

```
# Generated by qwen3-coder:480b-cloud
# Reviewed and improved by me
import numpy as np
import matplotlib.pyplot as plt
from scipy import stats

def shifted_exponential_samples(a, lam, n_samples=100):
    """
    Generate samples from shifted exponential distribution
    """
    standard_exp_samples = np.random.exponential(scale=1/lam, size=n_samples)
    samples = standard_exp_samples + a
    return samples

def shifted_exponential_pdf(x, a, lam):
    """
    Probability density function of shifted exponential distribution
    """
    pdf_values = np.where(x >= a, lam * np.exp(-lam * (x - a)), 0)
    return pdf_values

def mom_estimators(samples):
    """
    Method of Moments estimators for shifted exponential distribution

    For Shifted Exp(a, λ):
```

```
    E[X] = a + 1/λ
    Var[X] = 1/λ²

    MoM estimations:
    λ_hat = 1/sqrt(sample_variance)
    a_hat = sample_mean - 1/λ_hat
    """
    sample_mean = np.mean(samples)
    sample_var = np.var(samples)  # Population variance for MoM

    # MoM estimator: λ based on variance
    lam_mom = 1 / np.sqrt(sample_var)

    # MoM estimator: a based on λ_mom
    a_mom = sample_mean - 1 / lam_mom
    return a_mom, lam_mom

def mle_estimators(samples):
    """
    Maximum Likelihood Estimators for shifted exponential distribution

    MLE equations:
    â = min(X₁, X₂, ..., Xₙ) (minimum of samples)
    λ̂= n / Σ(Xᵢ - â)
    """
    n = len(samples)
    a_mle = np.min(samples)  # MLE of shift parameter
    lam_mle = n / np.sum(samples - a_mle)  # MLE of rate parameter

    return a_mle, lam_mle

# Parameters
true_a = 2.0        # true shift parameter
true_lam = 0.7      # true rate parameter
n_samples = 100
n_experiments = 5000

print("="*70)
print("SHIFTED EXPONENTIAL DISTRIBUTION")
print("="*70)
print(f"True parameters: a = {true_a}, λ = {true_lam}")
print(f"Sample size: {n_samples}")
print(f"Number of experiments: {n_experiments}")
print()

# Initialize arrays to store estimates
a_mom_estimates = np.zeros(n_experiments)
lam_mom_estimates = np.zeros(n_experiments)
a_mle_estimates = np.zeros(n_experiments)
lam_mle_estimates = np.zeros(n_experiments)

# Run experiments
for i in range(n_experiments):
    if (i + 1) % 1000 == 0:
```

```
        print(f"Completed {i + 1}/{n_experiments} experiments...")

    # Generate samples
    samples = shifted_exponential_samples(true_a, true_lam, n_samples)

    # Compute MoM estimators
    a_mom, lam_mom = mom_estimators(samples)
    a_mom_estimates[i] = a_mom
    lam_mom_estimates[i] = lam_mom

    # Compute MLE estimators
    a_mle, lam_mle = mle_estimators(samples)
    a_mle_estimates[i] = a_mle
    lam_mle_estimates[i] = lam_mle

print("All experiments completed!")
print()

# Calculate statistics
print("ESTIMATOR VALUES OVER 1 EXPERIMENTS:")
print("="*50)

print("Parameter 'a' estimators:")
print("-" * 30)
print(f"True value: {true_a}")
print(f"MoM: {a_mom_estimates[0]:.6f}")
print(f"MLE: {a_mle_estimates[0]:.6f}")
print()

print("Parameter 'λ' estimators:")
print("-" * 30)
print(f"True value: {true_lam}")
print(f"MoM: {lam_mom_estimates[0]:.6f}")
print(f"MLE: {lam_mle_estimates[0]:.6f}")
print()

# Create histograms
plt.figure(figsize=(15, 12))

# Histogram for 'a' parameter estimators
plt.subplot(1, 2, 1)
plt.axvline(true_a, color='green', linewidth=2, linestyle='--', label=f'True a =
{true_a}')
plt.axvline(np.mean(a_mom_estimates), color='cyan', linewidth=2, linestyle='--',
label=f'MoM mean = {np.mean(a_mom_estimates):.4f}')
plt.axvline(np.mean(a_mle_estimates), color='magenta', linewidth=2,
linestyle='--', label=f'MLE mean = {np.mean(a_mle_estimates):.4f}')
plt.hist(a_mom_estimates, bins=50, alpha=0.7, color='blue', edgecolor='black',
density=True, label='MoM')
plt.hist(a_mle_estimates, bins=50, alpha=0.7, color='red', edgecolor='black',
density=True, label='MLE')
# Get some kind of zoom inside interesting zones
plt.xlim(true_a - 0.2, true_a + 0.2)
plt.xlabel('Estimated value of a')
```

```
plt.ylabel('Density')
plt.title('Empirical Distribution of Estimator for Parameter a')
plt.legend()
plt.grid(True, alpha=0.3)

# Histogram for 'λ' parameter estimators
plt.subplot(1, 2, 2)
plt.axvline(true_lam, color='green', linewidth=2, linestyle='--', label=f'True λ =
{true_lam}')
plt.axvline(np.mean(lam_mom_estimates), color='cyan', linewidth=2, linestyle='--',
label=f'MoM mean = {np.mean(lam_mom_estimates):.4f}')
plt.axvline(np.mean(lam_mle_estimates), color='magenta', linewidth=2,
linestyle='--', label=f'MLE mean = {np.mean(lam_mle_estimates):.4f}')
plt.hist(lam_mom_estimates, bins=50, alpha=0.7, color='blue', edgecolor='black',
density=True, label='MoM')
plt.hist(lam_mle_estimates, bins=50, alpha=0.7, color='red', edgecolor='black',
density=True, label='MLE')
plt.xlabel('Estimated value of λ')
plt.ylabel('Density')
plt.title('Empirical Distribution of Estimator for Parameter λ')
plt.legend()
plt.grid(True, alpha=0.3)

plt.tight_layout()
plt.show()

# Summary comparison
print("SUMMARY COMPARISON:")
print("="*30)
print("For parameter 'a':")
print(f"  MoM Variance: {np.var(a_mom_estimates):.6f}")
print(f"  MLE Variance: {np.var(a_mle_estimates):.6f}")
print(f"  MLE has {(np.var(a_mom_estimates)/np.var(a_mle_estimates) - 1)*100:.1f}%
smaller variance")
print()
print("For parameter 'λ':")
print(f"  MoM Variance: {np.var(lam_mom_estimates):.6f}")
print(f"  MLE Variance: {np.var(lam_mle_estimates):.6f}")
print(f"  MLE has {(np.var(lam_mom_estimates)/np.var(lam_mle_estimates) -
1)*100:.1f}% smaller variance")
print()
```

6. Generate a sample of size $n = 100$ from the shifted exponential with parameters $(a^\star, \lambda^\star) = (0, 1)$, compute the two MoM estimators and the two MLE estimators, and report the values. Compare them and conclude.

```
Parameter 'a' estimators:
------------------------------
True value: 2.0
MoM: 2.193838
MLE: 2.006864

Parameter 'λ' estimators:
------------------------------
```

```
True value: 0.7
MoM: 0.795522
MLE: 0.692516
```

We can see that the MLE estimator is more accurate than the MoM estimator for both a and λ.

It is normal because the MLE estimator is based on a search of theoretical values that maximise the log-likelihood of the probability density function. It means that it will optimize the parameters to maximise the probability to have observed values near the given model. However the MoM estimator is simply based on correspondance between empirical and theoretical moments of the given model. In this way, since the objective is not to match the data to the model as closely as possible, the results are good, but not as good as those obtained with MLE, which attempts to match the data to the model as closely as possible by maximizing the likelihood of the data relative to the model.

7. Repeat the experiment 5000 times and produce two histograms:

- the empirical distribution of the estimator of a,
- the empirical distribution of the estimator of λ,

comparing Method of Moments and MLE. Print out the mean and variance of the two MoM estimators and the two MLE estimators over 5000 experiments.

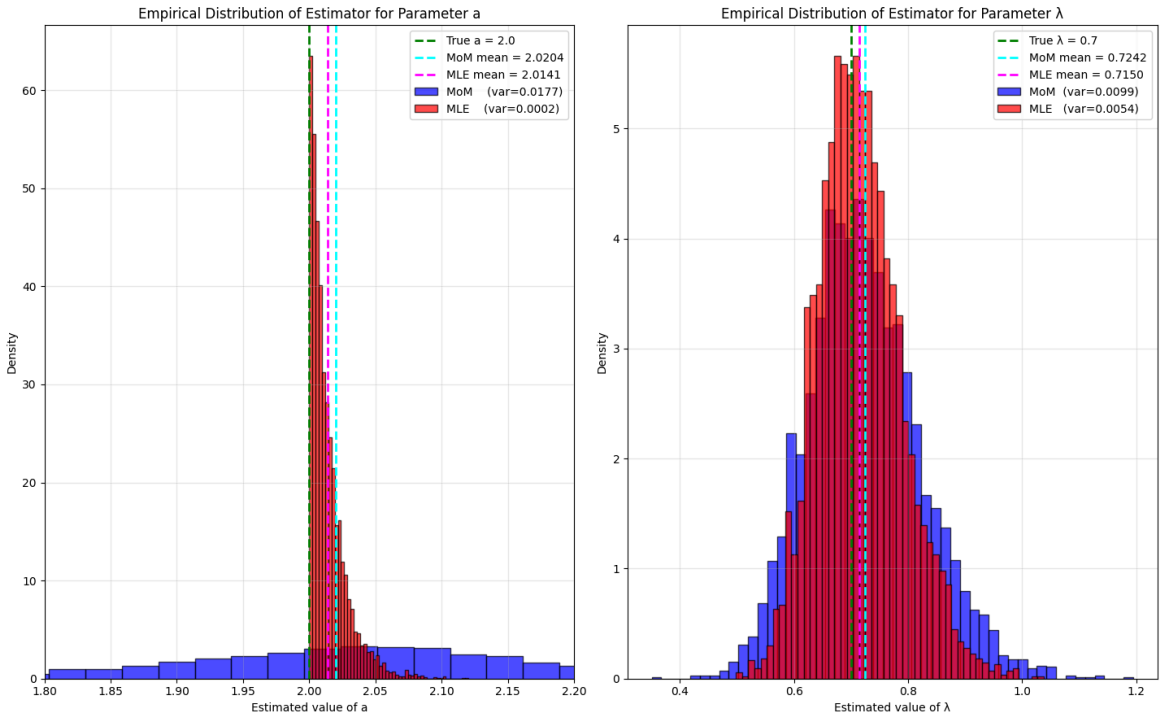


Figure 1: Result of the 5000 experiments

8. Interpret the results: which estimators appear more accurate? Which have lower variance? Give a short theoretical explanation for what you observe.

As we can see on Figure 1, in both case MLE has a smaller variance than MoM. Indeed, variance of $\hat{a}_{\text{MoM}}$ is 100 times bigger than variance of $\hat{a}_{\text{MLE}}$ and variance of $\hat{\lambda}_{\text{MoM}}$ is 2 times bigger than variance of $\hat{\lambda}_{\text{MLE}}$

Furthermore, in both cases, the mean obtained by MLE estimator is closer to the real value than the mean obtained by the MoM estimator.

This is due to the way MLE and MoM achieve estimations. MLE try to maximise the likelihood of the data relative to the model. On the contrary, MoM simply try to match the moments of the model with the empirical moments. In this way, some information is lost because only the two moments are taken into account, resulting in a loss of information contained in other moments of the model. This information is not lost during likelihood maximization, as this takes into account the entire likelihood function, which contains all the information contained in all moments of the model.

So the MLE estimator method is better than the MoM estimator method because more informations are taken into account to make the estimations.

MGFs : the Gaussian example

In Part 1 you used raw moments via empirical formulas; here you see how MGFs let you get the moments of an important distribution in closed form. For instance, moments of the Gaussian distribution are not easy to obtain from the integral definition, because Gaussian integrals do not have elementary antiderivatives and require several algebraic manipulations.

To illustrate the usefulness of MGFs as a theoretical tool, consider

$$X \sim N(\mu, \sigma^2), \quad M_{X(t)} = \exp\left(\mu t + \frac{1}{2}\sigma^2 t^2\right).$$

9. Using this MGF, compute $\mathbb{E}[X]$ and $\mathbb{E}[X^2]$ by differentiating $M_{X(t)}$ at $t = 0$, and verify that the variance of X is σ^2. (You may attempt the integrals directly to see why the MGF method is much simpler, but this is not required.)

   First, we will evaluate the first derivate of $M_{X(t)}$ according t and evaluate it when $t = 0$ to obtain $\mathbb{E}[X]$.

   We have:

$$\begin{aligned}\frac{\partial M_{X(t)}}{\partial t} &= \frac{\partial}{\partial t}\exp\left(\mu t + \frac{1}{2}\sigma^2 t^2\right) \\ &= (\mu + \sigma^2 t)\exp\left(\mu t + \frac{1}{2}\sigma^2 t^2\right)\end{aligned}$$

   So when $t = 0$, we have:

$$\begin{aligned}\mathbb{E}[X] &= (\mu + \sigma^2 \cdot 0)\exp\left(\mu \cdot 0 + \frac{1}{2}\sigma^2 0^2\right) \\ &= \mu \exp(0) \\ &= \mu\end{aligned}$$

   Now, to find the second moment of the function, we need to take the second derivate of $M_{X(t)}$ and evaluate it when $t = 0$.

   We have:

$$\begin{aligned}
\frac{\partial^2 M_{X(t)}}{\partial t^2} &= \frac{\partial}{\partial t}(\mu + \sigma^2 t)\exp\left(\mu t + \frac{1}{2}\sigma^2 t^2\right) \\
&= \frac{\partial}{\partial t}\mu \exp\left(\mu t + \frac{1}{2}\sigma^2 t^2\right) + \frac{\partial}{\partial t}\sigma^2 t \exp\left(\mu t + \frac{1}{2}\sigma^2 t^2\right) \\
&= \mu(\mu + \sigma^2 t)\exp\left(\mu t + \frac{1}{2}\sigma^2 t^2\right) + \sigma^2 \exp\left(\mu t + \frac{1}{2}\sigma^2 t^2\right) + \sigma^2 t(\mu + \sigma^2 t)\exp\left(\mu t + \frac{1}{2}\sigma^2 t^2\right) \\
&= \left((\mu + \sigma^2 t)^2 + \sigma^2\right)\exp\left(\mu t + \frac{1}{2}\sigma^2 t^2\right)
\end{aligned}$$

So when $t = 0$, we have:

$$\begin{aligned}
\mathbb{E}[X^2] &= ((\mu + 0)^2 + \sigma^2)\exp(0) \\
&= \mu^2 + \sigma^2
\end{aligned}$$

So the variance give us:

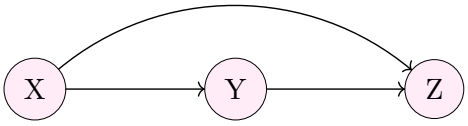
$$\begin{aligned}
\mathbb{E}[(X - \mathbb{E}[X])^2] &= \mathbb{E}[X^2 - 2\mathbb{E}[X]X + \mathbb{E}[X]^2] \\
&= \mathbb{E}[X^2] - 2\mathbb{E}[X]\mathbb{E}[X] + \mathbb{E}[X]^2 \\
&= \mathbb{E}[X^2] - \mathbb{E}[X]^2 \\
&= \mu^2 + \sigma^2 - \mu^2 \\
&= \sigma^2
\end{aligned}$$

10. Explain when MGFs can be useful.

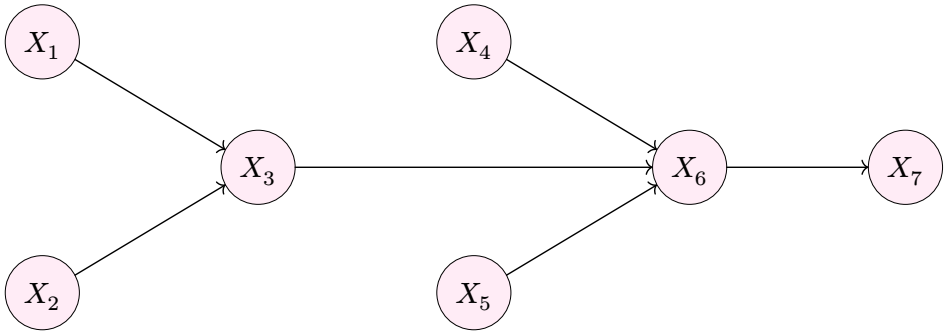
As we can see, the MGFs is very useful to easily compute the moments of a function. So it allows to easily extract each moments of the function and modelise the function with only a few moments. So MGFs is very useful to simplify the real function and theoretical proof by simplifying a lot the computations, avoiding complex integrations.

Chain rules for probabilities

For each of the 3 following chains of information, give the joint probability using the chain rule:



1.
   - $p(X, Y, Z) = p(X) \cdot p(Y \mid X) \cdot p(Z \mid X, Y)$
   - $p(X, Z) = p(X) \cdot p(Z \mid X)$
   - $p(X, Y) = p(X) \cdot p(Y \mid X)$
   - $p(Y, Z) = p(Y) \cdot p(Z \mid Y)$



2.
   - $p(X_1, X_2, X_3, X_4, X_5, X_6, X_7) = p(X_1) \cdot p(X_2) \cdot p(X_3 \mid X_1, X_2) \cdot p(X_4) \cdot p(X_5) \cdot p(X_6 \mid X_5, X_4, X_3) \cdot p(X_7 \mid X_6)$
   - $p(X_1, X_3, X_5, X_7) = \sum_{X_2} \sum_{X_4} \sum_{X_6} p(X_1, ..., X_7) = p(X_1) \cdot p(X_3 \mid X_1) \cdot p(X_5) \cdot p(X_7 \mid X_5, X_3)$
   - $p(X_2, X_4, X_6, X_7) = p(X_2) \cdot p(X_4) \cdot p(X_6 \mid X_2, X_4) \cdot p(X_7 \mid X_6)$
   - $p(X_3, X_6, X_7) = p(X_3) \cdot p(X_6 \mid X_3) \cdot p(X_7 \mid X_6)$
   - $p(X_1, X_2, X_4, X_5) = p(X_1) \cdot p(X_2) \cdot p(X_4) \cdot p(X_5)$

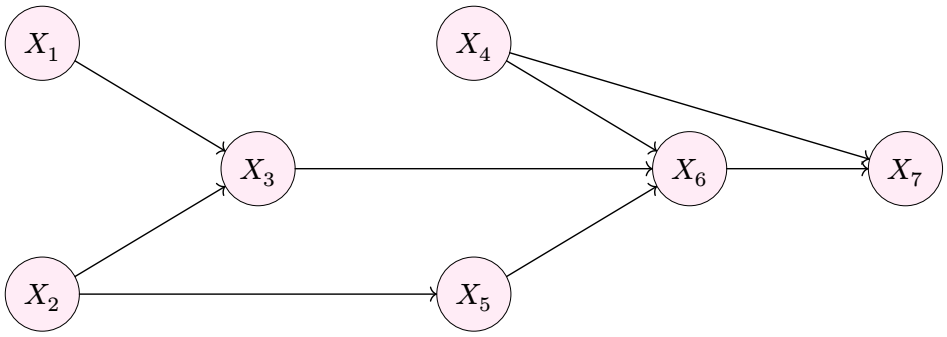


Figure 4: Complex graphical model for X_1 to X_7

3.
   - $p(X_1, X_2, X_3, X_4, X_5, X_6, X_7) = p(X_1) \cdot p(X_2) \cdot p(X_3 \mid X_1, X_2) \cdot p(X_4) \cdot p(X_5 \mid X_2) \cdot p(X_6 \mid X_5, X_4, X_3) \cdot p(X_7 \mid X_6, X_4)$
   - $p(X_1, X_3, X_5, X_7) = \sum_{X_2} \sum_{X_4} \sum_{X_6} p(X_1, ..., X_7) = p(X_1) \cdot p(X_3 \mid X_1) \cdot p(X_5) \cdot p(X_7 \mid X_5, X_3)$
   - $p(X_2, X_4, X_6, X_7) = p(X_2) \cdot p(X_4) \cdot p(X_6 \mid X_4, X_2) \cdot p(X_7 \mid X_6, X_4)$
   - $p(X_3, X_6, X_7) = p(X_3) \cdot p(X_6 \mid X_3) \cdot p(X_7 \mid X_6)$

- $p(X_1, X_2, X_4, X_5) = p(X_1) \cdot p(X_2) \cdot p(X_4) \cdot p(X_5 \mid X_2)$

Problem: information quantifiers

U, V and W are three binary random variables. Their joint probability mass function is depicted as below:

U	V	W	$p_{U,V,W}(u,v,w)$
0	0	0	$\frac{1}{4}$
0	0	1	0
0	1	0	$\frac{1}{4}$
0	1	1	$\frac{1}{8}$
1	0	0	0
1	0	1	$\frac{1}{8}$
1	1	0	0
1	1	1	$\frac{1}{4}$

Calculate the information measures below:

1. $H(U)$, $H(V)$ and $H(W)$
   - $p_{U(u=0)} = \sum_{v\in V}\sum_{w\in W} p_{U,V,W}(u=0,v,w) = \frac{1}{4} + 0 + \frac{1}{4} + \frac{1}{8} = \frac{5}{8}$
   - $p_{U(u=1)} = 1 - p_{U(u=0)} = 1 - \frac{5}{8} = \frac{3}{8}$
   - $p_{V(v=0)} = \sum_{u\in U}\sum_{w\in W} p_{U,V,W}(u,v=0,w) = \frac{1}{4} + 0 + 0 + \frac{1}{8} = \frac{3}{8}$
   - $p_{V(v=1)} = 1 - p_{V(v=0)} = 1 - \frac{3}{8} = \frac{5}{8}$
   - $p_{W(w=0)} = \sum_{u\in U}\sum_{v\in V} p_{U,V,W}(u,v,w=0) = \frac{1}{4} + \frac{1}{4} + 0 + 0 = \frac{1}{2}$
   - $p_{W(w=1)} = 1 - p_{W(w=0)} = 1 - \frac{1}{2} = \frac{1}{2}$

   So we have:

   - $H(U) = -\sum_{u\in U} p_{U(u)} \log_2\big(p_{U(u)}\big) = -\frac{5}{8}\log_2\big(\frac{5}{8}\big) - \frac{3}{8}\log_2\big(\frac{3}{8}\big) \approx 0.95$
   - $H(V) = -\sum_{v\in V} p_{V(v)} \log_2\big(p_{V(v)}\big) = -\frac{5}{8}\log_2\big(\frac{5}{8}\big) - \frac{3}{8}\log_2\big(\frac{3}{8}\big) \approx 0.95$
   - $H(W) = -\sum_{w\in W} p_{W(w)} \log_2\big(p_{W(w)}\big) = -\frac{1}{2}\log_2\big(\frac{1}{2}\big) - \frac{1}{2}\log_2\big(\frac{1}{2}\big) = 1$

2. $H(U \mid V)$, $H(V \mid U)$ and $H(W \mid U)$
   - $$\begin{aligned} H(U \mid V) &= H(U,V) - H(V) \\ &= -\sum_{u\in U}\sum_{v\in V} p_{U,V}(u,v)\log_2\big(p_{U,V}(u,v)\big) - H(V) \\ &= -\tfrac{1}{4}\log_2\big(\tfrac{1}{4}\big) - \tfrac{3}{8}\log_2\big(\tfrac{3}{8}\big) - \tfrac{1}{8}\log_2\big(\tfrac{1}{8}\big) - \tfrac{1}{4}\log_2\big(\tfrac{1}{4}\big) - H(V) \\ &\approx 1.9 - 0.95 \\ &\approx 0.95 \end{aligned}$$
   - $$\begin{aligned} H(V \mid U) &= H(U,V) - H(U) \\ &= H(U \mid V) + H(V) - H(U) \\ &\approx 0.95 + 0.95 - 0.95 \\ &\approx 0.95 \end{aligned}$$

- $H(U \mid W) = H(U,W) - H(W)$
  $= -\sum_{u\in U}\sum_{w\in W} p_{U,W}(u,w)\log_2\big(p_{U,W}(u,w)\big) - H(W)$
  $= -\frac{1}{2}\log_2\big(\frac{1}{2}\big) - \frac{1}{8}\log_2\big(\frac{1}{8}\big) - 0 - \frac{3}{8}\log_2\big(\frac{3}{8}\big) - 1$
  $\approx 1.4 - 1$
  ≈ 0.4

3. $I(U;V)$, $I(U;W)$ and $I(V;W)$
   - $I(U;V) = H(U) - H(U \mid V)$
     $\approx 0.95 - 0.95$
     ≈ 0
   - $I(U;W) = H(U) - H(U \mid W)$
     $\approx 0.95 - 0.4$
     ≈ 0.55
   - $I(V;W) = H(V) + H(W) - H(V \mid W)$
     $= H(V) + H(W) - (H(V,W) - H(W))$
     $= H(V) + 2H(W) - H(V,W)$
     $= H(V) + 2H(W) + \sum_{v\in V}\sum_{w\in W} p_{V,W}(v,w)\log_2\big(p_{V,W}(v,w)\big)$
     $= H(V) + 2H(W) + \frac{1}{4}\log_2\big(\frac{1}{4}\big) + \frac{1}{8}\log_2\big(\frac{1}{8}\big) + \frac{1}{4}\log_2\big(\frac{1}{4}\big) + \frac{3}{8}\log_2\big(\frac{3}{8}\big)$
     $\approx 0.95 + 2 - 1.9$
     ≈ 1.05

4. $H(U,V,W)$

   We have:

$$\begin{aligned} H(U,V,W) &= -\sum_{u\in U}\sum_{v\in V}\sum_{w\in W} p_{U,V,W}(u,v,w)\log_2\big(p_{U,V,W}(u,v,w)\big) \\ &= -3\cdot\frac{1}{4}\log_2\left(\frac{1}{4}\right) - 2\cdot\frac{1}{8}\log_2\left(\frac{1}{8}\right) \\ &= \frac{3}{2} + \frac{3}{4} \\ &= 2.25 \end{aligned}$$

   For each of the above items you could directly use the definition. However, because they are related to each other in many ways, you could calculate some of them and derive the rest by using their relations: chain rules for entropy and mutual information, the Venn diagrams.

Problem: Source Coding

We study a binary memoryless source X, where

$$P(X=1) = \theta = 0.1, \quad P(X=0) = 0.9.$$

Tasks

1. Data Generation:
   - Generate $n = 10,000$ binary samples.
   - Group them into symbols of length 5 (2000 total symbols).

- Count the frequency of each symbol.

```
import numpy as np
from dahuffman import HuffmanCodec

samples = np.random.rand(2000, 5)
samples[samples >= 0.9] = 1
samples[samples < 0.9] = 0
samples = ["".join(str(int(elem)) for elem in row) for row in samples]

symbols, frequencies = np.unique(samples, axis=0, return_counts=True)
print(f"Frequency of each symbol: {frequencies}")
```

```
Frequency of each symbol: [1185  140  113   13  137   18   13  130   16   11
2   11    3  133  18   17    1   17    2    2   13    1    1    3]
```

2. Huffman Coding:
   - Build a Huffman code (e.g. with `dahuffman`) using the symbol frequencies.
   - Encode the sequence and record its total length in bits.

```
codec = HuffmanCodec.from_frequencies(dict(zip(symbols, list(frequencies))))
encoded = codec.encode(samples)
print(f"Number of bits used for encoding: {len(encoded)}")
```

```
Number of bits used for encoding: 594
```

3. Entropy and Efficiency:
   - Compute the theoretical entropy $H(X)$. $H(X) = -\sum_X P(X)\log_2(P(X)) = -0.1\log_2(0.1) - 0.9\log_2(0.9) \approx 0.468995593$
   - Compute the average Huffman code length:

$$L = \frac{\text{encoded length (bits)}}{\text{number of symbols}} = \frac{594}{2000} = 0.297$$

   - Compare $H(X)$ and L. Comment on compression efficiency.

     Here the entropy indicate the number of bits needed to encode the variable created by the probability rule. So it means that we need an average of 0.469 bits per binary sample to encode the most efficiently the binary sequence in a brute force way. However, if we look at the Huffman coding, we can constate that only 0.297 bits are needed to encode a binary sample. So it means that the Huffman coding, by switching the representation of the sequence in a more intelligent way using frequencies of each symbols, allows a compression of the sequence. Indeed the efficiency of the Huffman coding with only 0.297 bits needed for a sample is better than the base entropy of 0.469 bits per sample.

     So it is very important for compression to use a good representation of the data in such a way an intelligent encoding can be achieved to reduce number of bits needed to properly encode the data.

(optional) Problem: Communication System with Noise

We send a binary sequence through a channel with additive white Gaussian noise (AWGN).

System Setup

- A '1' is transmitted as amplitude $A = 1$, a '0' as $A = 0$.

- Each bit is repeated N times ($N = 10$ at first).
- The channel adds Gaussian noise $z[n] \sim N(0, \sigma_z^2)$ with $\sigma_z^2 = 1.5$.

Tasks

1. Signal Generation: Generate $k = 10,000$ random bits, build the transmitted signal $x[n]$, and add AWGN to get $y[n]$.

```python
import numpy as np

random_bits = np.random.randint(0, 2, size=(10000, 1))
signal = random_bits @ np.ones(10)
signal = signal.ravel()

noise = np.random.normal(loc=0, scale=np.sqrt(1.5), size=signal.shape[0])
transmitted_signal = (signal + noise).astype(int)
```

2. Detection Rules:
   - Derive the Neyman-Pearson rule with false alarm $P_{\text{FA}} = 0.01$.

     We consider that $x[n]$ is the signal to transmit and $z[n]$ is the noise added to the signal, that give us $y[n] = x[n] + z[n]$.

     If we consider H_0 as the hypothesis "bit 0 is transmitted" and H_1 as the hypothesis "bit 1 is transmitted", we have

$$P(y[n]; H_0) = \underbrace{x[n]}_{=0 \text{ due to } H_0} + z[n] = z[n] = \frac{1}{\sqrt{2\pi\sigma_z^2}} \exp^{-\frac{(y[n]-0)^2}{2\sigma_z^2}}$$

     and

$$P(y[n]; H_1) = \underbrace{x[n]}_{=1 \text{ due to } H_1} + z[n] = \frac{1}{\sqrt{2\pi\sigma_z^2}} \frac{\exp^{-(y[n]-1)^{2^1}}}{2\sigma_z^2})$$

     According to the definition of Neyman-Pearson, to maximise P_D for a given $P_{FA} = \alpha$, decide H_1, if:

$$L(x) = \frac{p(x; H_1)}{p(x; H_0)} > \gamma$$

     where the threshold γ is found from:

$$P_{FA} = \int_{x:L(x)>\gamma} p(x; H_0)dx = \alpha$$

     In our case, we have $P_{FA} = 0.01$ and $x = y[n]$. So we have:

[1] Here we have $y[n] - 1$ because we add some gaussian noise $z[n] \sim N(0, \sigma_z^2)$ to an amplitude of 1 meaning that the mean of the resulting distribution is 1

$$
\begin{aligned}
L(x) = \frac{P(y[n]; H_1)}{P(y[n]; H_0)} &> \gamma \\
&= \frac{\frac{1}{\sqrt{2\pi\sigma_z^2}} \exp^{-\frac{(y[n]-1)^2}{2\sigma_z^2}}}{\frac{1}{\sqrt{2\pi\sigma_z^2}} \exp^{-\frac{y[n]^2}{2\sigma_z^2}}} > \gamma \\
&= \exp^{\frac{y[n]^2}{2\sigma_z^2} - \frac{(y[n]-1)^2}{2\sigma_z^2}} > \gamma \\
&= \exp^{\frac{1}{2\sigma_z^2}(y[n]^2 - y[n]^2 + 2y[n] - 1)} > \gamma \\
&= \exp^{\frac{1}{2\sigma_z^2}(2y[n]-1)} > \gamma \\
&= \exp^{\frac{2y[n]-1}{3}} > \gamma
\end{aligned}
$$

- Derive the Bayesian rule assuming $P(H_0) = P(H_1) = 0.5$.
- Report both thresholds γ_{NP} and γ_{Bayes}. Compare them.

3. Apply Detection: Use both rules to estimate the received bits. Compare with the original sequence.

Write your solution here

4. Visualization: Plot $x[n]$, $y[n]$, and the estimated $\hat{x}[n]$. Zoom in on 100 samples to see the noise effect.

Write your solution here

5. Performance: Compute empirical P_M (miss), P_{FA} (false alarm), and P_e (total error). Compare with theoretical values.

Write your solution here

6. Effect of N: Vary N from 1 to 100. Plot how P_M, P_{FA}, and P_e change for both detection rules. Explain why larger N improves detection.

Write your solution here

(Small project (optional)) When moment matching fails

In this section we study a simple nonlinear simulator and investigate how different moment-matching losses behave in practice.

Consider the generator

$$X = g_{\theta(Z)} = \theta_0 + \theta_1 Z^2 + \theta_2 Z^5, \quad Z \sim N(0,1),$$

with true parameters

$$\theta^\star = (2, 0.7, 0.05).$$

Data generation

Generate $n = 10,000$ samples $X_1, ..., X_n$ from the model using the true parameters. Keep these samples fixed for all subsequent experiments (i.e. do not regenerate new data each iteration).

(b) Optimization

Let $\theta = (\theta_0, \theta_1, \theta_2)$ be a trainable parameter vector in PyTorch, initialized at

$$\theta^{(0)} = (1.6, 0.7, 0.01).$$

Use the Adam optimizer with learning rate 10^{-3}. At each iteration, draw a fresh batch $Z \sim N(0,1)$ of size 1024 and compute $X_{\text{model}} = g_{\theta(Z)}$.

Unless stated otherwise, run all experiments below for $10,000$ iterations.

Train the parameter vector θ using the following losses, one at a time:

1. **Match only the mean**:

$$L_1 = (\mathbb{E}[X_{\text{model}}] - \mathbb{E}[X_{\text{data}}])^2.$$

2. **Match mean and variance**:

$$L_2 = (\mathbb{E}[X_{\text{model}}] - \mathbb{E}[X_{\text{data}}])^2 + (\text{Var}(X_{\text{model}}) - \text{Var}(X_{\text{data}}))^2.$$

3. **Match mean, variance, and 5th central moment**:

$$L_3 = L_2 + \Big(\mathbb{E}\big[(X_{\text{model}} - \mathbb{E}[X_{\text{model}}])^5\big] - \mathbb{E}\big[(X_{\text{data}} - \mathbb{E}[X_{\text{data}}])^5\big]\Big)^2.$$

After training with each loss, record the final value of θ.

Kernelized moment matching (MMD)

Define the Gaussian kernel

$$k(x,y) = \exp\left(-\frac{(x-y)^2}{2\sigma^2}\right), \quad \sigma = 2.$$

Define the empirical squared MMD between two batches x and y as

$$\text{MMD}^2(x,y) = \frac{1}{m^2}\sum_{i,j} k(x_i, x_j) + \frac{1}{m^2}\sum_{i,j} k(y_i, y_j) - \frac{2}{m^2}\sum_{i,j} k(x_i, y_j).$$

```
def rbf_kernel(x, y, sigma=1.0):
    x = x.view(-1, 1)
    y = y.view(1, -1)
    diff2 = (x - y)**2
    return torch.exp(-diff2 / (2 * sigma**2))

def mmd_squared(x, y, sigma=1.0):
    Kxx = rbf_kernel(x, x, sigma)
    Kyy = rbf_kernel(y, y, sigma)
    Kxy = rbf_kernel(x, y, sigma)
    m = x.shape[0]
    n = y.shape[0]
    return Kxx.mean() + Kyy.mean() - 2 * Kxy.mean()
```

Now train θ by minimizing

$$L_{\text{MMD}} = \text{MMD}^2(X_{\text{data}}, X_{\text{model}}).$$

Record the final value of θ.

Empirical comparison

For each loss L_1, L_2, L_3, and L_{MMD}:

Generate $10,000$ new samples from $g_{\theta(Z)}$ using the trained parameter vector. Plot a histogram of this model-generated sample along with a histogram of the real data sample. Compare the fitted parameter vector θ with the true value $\theta^\star$.

```
_, bins, _ = plt.hist(
    x_data.numpy(), bins=50, density=True,
    alpha=0.5, label="data"
)
plt.hist(
    x_model_final.numpy(), bins=bins, density=True,
    alpha=0.5, label="model (MMD trained)"
)
```

Interpretation

Provide a concise discussion:

- Which losses successfully recover (θ_0, θ_1)?
- What happens when including the 5th central moment? Why does this lead to unstable training or incorrect θ_2?
- Why does the MMD loss (implicitly matching infinitely many smooth bounded moments) produce stable and accurate estimates?
- How does this illustrate the general difference between matching raw high-order moments and using smooth kernel-based moments?